DSA ASSIGNMENT

NAME: V.Yaswanth
ROLL NO: A25126510062
SEC: II/IV CSE-A

UNIT-1

Q1) A company stores employee IDs in ascending order. Write a C program that accepts n employee IDs, searches for a required ID using Binary Search, displays its position when found, reports when it is absent, and counts the number of comparisons. Test the program for both successful and unsuccessful searches.

SOL)

```
#include <stdio.h>

int main() {
    int n, key, low, high, mid;
    int comparisons = 0;
    int found = 0;

    printf("Enter the number of employee IDs: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid input for number of employees.\n");
        return 1;
    }

    int emp_ids[n];
    printf("Enter %d employee IDs in ascending order:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &emp_ids[i]);
    }

    printf("Enter the employee ID to search: ");
    scanf("%d", &key);

    low = 0;
    high = n - 1;

    while (low <= high) {
        comparisons++;
        mid = low + (high - low) / 2;
        if (emp_ids[mid] == key) {
            found = 1;
            break;
        } else if (emp_ids[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
```

```
    }

    if (found) {
        printf("\nStatus: Success (ID found)\n");
        printf("Position: %d (0-indexed: %d)\n", mid + 1, mid);
        printf("Total comparisons: %d\n", comparisons);
    } else {
        printf("\nStatus: Failure (ID not found)\n");
        printf("Total comparisons: %d\n", comparisons);
    }

    return 0;
}
```

OUTPUT:

```
PS E:\cpp programs> cd "e:\cpp programs\" ; if ($?) { gcc test.c -o test } ; if ($?) { .\test }
Enter the number of employee IDs: 6
Enter 6 employee IDs in ascending order:
101 105 110 125 140 155
Enter the employee ID to search: 125

Status: Success (ID found)
Position: 4 (0-indexed: 3)
Total comparisons: 3
```

Q2) A teacher wants to arrange student marks in ascending order and also measure how much rearrangement is necessary. Write a C program using Insertion Sort that accepts n marks, displays the array after every pass, counts the total number of element shifts, and displays the final sorted list and shift count.

SOL)

```
#include <stdio.h>

int main() {
    int n, i, j, key;
    int shifts = 0;

    printf("Enter the number of student marks: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid input for number of marks.\n");
        return 1;
    }

    int marks[n];
    printf("Enter %d student marks:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &marks[i]);
    }

    printf("\nInitial array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", marks[i]);
    }

    printf("\n\n--- Sorting Passes ---\n");
    for (i = 1; i < n; i++) {
        key = marks[i];
        j = i - 1;
        int shifted_in_pass = 0;

        while (j >= 0 && marks[j] > key) {
            marks[j + 1] = marks[j];
            shifts++;
            shifted_in_pass++;
```

```
            j--;
        }
        marks[j + 1] = key;

        printf("Pass %d: ", i);
        for (int k = 0; k < n; k++) {
            printf("%d ", marks[k]);
        }
        printf("(Shifts in this pass: %d)\n", shifted_in_pass);
    }

    printf("\n--- Final Results ---\n");
    printf("Sorted marks in ascending order: ");
    for (i = 0; i < n; i++) {
        printf("%d ", marks[i]);
    }
    printf("\nTotal number of element shifts: %d\n", shifts);

    return 0;
}
```

OUTPUT:

```
PS E:\cpp programs> cd "e:\cpp programs\" ; if ($?) { gcc test.c -o test } ; if ($?) { .\test }
Enter the number of student marks: 5
Enter 5 student marks:
45 12 85 32 10

Initial array: 45 12 85 32 10

--- Sorting Passes ---
Pass 1: 12 45 85 32 10 (Shifts in this pass: 1)
Pass 2: 12 45 85 32 10 (Shifts in this pass: 0)
Pass 3: 12 32 45 85 10 (Shifts in this pass: 2)
Pass 4: 10 12 32 45 85 (Shifts in this pass: 4)

--- Final Results ---
Sorted marks in ascending order: 10 12 32 45 85
Total number of element shifts: 7
```

UNIT 2

Q1) An expression-processing application receives an arithmetic expression in infix form. Write a C program using a stack to convert it to postfix form while correctly handling parentheses and operator precedence for +, -, *, / and ^. Test the program using an expression containing multiple operators and parentheses.

SOL)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
   if (top < MAX - 1) {
      stack[++top] = c;
   }
}

char pop() {
   if (top >= 0) {
      return stack[top--];
   }
   return -1;
}

char peek() {
   if (top >= 0) {
      return stack[top];
   }
   return -1;
}

int precedence(char c) {
   if (c == '^')
      return 3;
   else if (c == '*' || c == '/')
      return 2;
   else if (c == '+' || c == '-')
      return 1;
   else
      return -1;
}

int isRightAssociative(char c) {
   if (c == '^')
      return 1;
   return 0;
}

void infixToPostfix(char* infix, char* postfix) {
   int i, j = 0;
   int len = strlen(infix);

   for (i = 0; i < len; i++) {
      char c = infix[i];

      if (isalnum(c)) {
         postfix[j++] = c;
```

```
        } else if (c == '(') {
            push(c);
        } else if (c == ')') {
            while (top != -1 && peek() != '(') {
                postfix[j++] = pop();
            }
            if (top != -1 && peek() == '(') {
                pop();
            }
        } else {
            while (top != -1 && precedence(peek()) >= precedence(c)) {
                if (c == '^' && peek() == '^' && isRightAssociative(c)) {
                    break;
                }
                postfix[j++] = pop();
            }
            push(c);
        }
    }

    while (top != -1) {
        postfix[j++] = pop();
    }
    postfix[j] = '\0';
}

int main() {
    char infix[MAX];
    char postfix[MAX];

    printf("Enter an infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("Postfix expression: %s\n", postfix);

    return 0;
}
```

OUTPUT:

```
PS E:\cpp programs> cd "e:\cpp programs\" ; if ($?) { gcc test.c -o test } ; if ($?) { .\test }
Enter an infix expression: A*(B+C)/D^E
Postfix expression: ABC+*DE^/
```

Q2) A service centre uses a fixed-size request buffer in which released positions must be reused. Write a C program to implement a Circular Queue using an array with insertion, deletion, display, overflow and underflow operations. Demonstrate that positions freed after deletion can be reused for new requests.

SOL)

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1;
int rear = -1;

int isFull() {
    if ((front == 0 && rear == SIZE - 1) || (front == rear + 1)) {
        return 1;
    }
    return 0;
}
```

```
int isEmpty() {
    if (front == -1) {
        return 1;
    }
    return 0;
}

void insert(int value) {
    if (isFull()) {
        printf("Overflow! Request buffer is full. Cannot insert %d.\n", value);
        return;
    }
    if (front == -1) {
        front = 0;
        rear = 0;
    } else if (rear == SIZE - 1 && front != 0) {
        rear = 0;
    } else {
        rear++;
    }
    queue[rear] = value;
    printf("Successfully inserted request ID: %d\n", value);
}

void delete() {
    if (isEmpty()) {
        printf("Underflow! Request buffer is empty. No request to process.\n");
        return;
    }
    printf("Processed and removed request ID: %d (Position %d freed)\n", queue[front], front);
    if (front == rear) {
        front = -1;
        rear = -1;
    } else if (front == SIZE - 1) {
        front = 0;
    } else {
        front++;
    }
}

void display() {
    if (isEmpty()) {
        printf("Request buffer is currently empty.\n");
        return;
    }
    printf("Current Request Buffer: ");
    int i = front;
    if (front <= rear) {
        while (i <= rear) {
            printf("%d ", queue[i]);
            i++;
        }
    } else {
        while (i < SIZE) {
            printf("%d ", queue[i]);
            i++;
        }
        i = 0;
        while (i <= rear) {
```

```
            printf("%d ", queue[i]);
            i++;
        }
    }
    printf("\nIndices -> Front: %d, Rear: %d\n", front, rear);
}

int main() {
    printf("--- Initializing Circular Queue of Size %d ---\n\n", SIZE);

    printf("Step 1: Fill the queue completely\n");
    insert(101);
    insert(102);
    insert(103);
    insert(104);
    insert(105);
    display();

    printf("\nStep 2: Attempt overflow insertion when full\n");
    insert(106);

    printf("\nStep 3: Delete two items to free positions at the beginning\n");
    delete();
    delete();
    display();

    printf("\nStep 4: Reuse freed positions by inserting new requests\n");
    insert(106);
    insert(107);
    display();

    printf("\nStep 5: Attempt overflow again with wrapped-around queue\n");
    insert(108);

    printf("\nStep 6: Empty the queue completely to test underflow\n");
    delete();
    delete();
    delete();
    delete();
    delete();
    delete(); // Triggers underflow

    return 0;
}
```

OUTPUT:

```
PS E:\cpp programs> cd "e:\cpp programs\" ; if ($?) { gcc test.c -o test } ; if ($?) { .\test }
--- Initializing Circular Queue of Size 5 ---

Step 1: Fill the queue completely
Successfully inserted request ID: 101
Successfully inserted request ID: 102
Successfully inserted request ID: 103
Successfully inserted request ID: 104
Successfully inserted request ID: 105
Current Request Buffer: 101 102 103 104 105
Indices -> Front: 0, Rear: 4

Step 2: Attempt overflow insertion when full
Overflow! Request buffer is full. Cannot insert 106.

Step 3: Delete two items to free positions at the beginning
Processed and removed request ID: 101 (Position 0 freed)
Processed and removed request ID: 102 (Position 1 freed)
Current Request Buffer: 103 104 105
Indices -> Front: 2, Rear: 4

Step 4: Reuse freed positions by inserting new requests
Successfully inserted request ID: 106
Successfully inserted request ID: 107
Current Request Buffer: 103 104 105 106 107
Indices -> Front: 2, Rear: 1

Step 5: Attempt overflow again with wrapped-around queue
Overflow! Request buffer is full. Cannot insert 108.

Step 6: Empty the queue completely to test underflow
Processed and removed request ID: 103 (Position 2 freed)
Processed and removed request ID: 104 (Position 3 freed)
Processed and removed request ID: 105 (Position 4 freed)
Processed and removed request ID: 106 (Position 0 freed)
```

UNIT 3

Q1) A department maintains student roll numbers dynamically. Write a C program using a Singly Linked List to create the list, insert at the beginning and end, search for a specified roll number, delete a specified roll number, and display the updated list after each operation. Handle the case when a requested roll number is not available.

SOL)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int roll_no;
   struct Node* next;
};

struct Node* head = NULL;

void display() {
   struct Node* temp = head;
   if (temp == NULL) {
      printf("The roll number list is currently empty.\n");
      return;
   }
   printf("Current Roll Numbers: ");
   while (temp != NULL) {
      printf("%d -> ", temp->roll_no);
      temp = temp->next;
   }
   printf("NULL\n");
}

void insertAtBeginning(int roll) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->roll_no = roll;
   newNode->next = head;
   head = newNode;
   printf("Inserted %d at the beginning.\n", roll);
   display();
}

void insertAtEnd(int roll) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->roll_no = roll;
   newNode->next = NULL;
   if (head == NULL) {
      head = newNode;
   } else {
      struct Node* temp = head;
      while (temp->next != NULL) {
         temp = temp->next;
      }
      temp->next = newNode;
   }
   printf("Inserted %d at the end.\n", roll);
   display();
}

void searchRoll(int roll) {
   struct Node* temp = head;
   int position = 1;
   while (temp != NULL) {
```

```
        if (temp->roll_no == roll) {
            printf("Roll number %d found at position %d in the list.\n", roll, position);
            return;
        }
        temp = temp->next;
        position++;
    }
    printf("Roll number %d is not available in the list.\n", roll);
}

void deleteRoll(int roll) {
    struct Node* temp = head;
    struct Node* prev = NULL;

    if (temp != NULL && temp->roll_no == roll) {
        head = temp->next;
        free(temp);
        printf("Deleted roll number %d from the list.\n", roll);
        display();
        return;
    }

    while (temp != NULL && temp->roll_no != roll) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Roll number %d not found. Deletion failed.\n", roll);
        return;
    }

    prev->next = temp->next;
    free(temp);
    printf("Deleted roll number %d from the list.\n", roll);
    display();
}

int main() {
    int choice, roll;

    while (1) {
        printf("\n--- Student Roll Number Management Menu ---\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Search Roll Number\n");
        printf("4. Delete Roll Number\n");
        printf("5. Display List\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Exiting.\n");
            break;
        }

        switch (choice) {
            case 1:
                printf("Enter roll number to insert at beginning: ");
                scanf("%d", &roll);
```

```
                insertAtBeginning(roll);
                break;
            case 2:
                printf("Enter roll number to insert at end: ");
                scanf("%d", &roll);
                insertAtEnd(roll);
                break;
            case 3:
                printf("Enter roll number to search: ");
                scanf("%d", &roll);
                searchRoll(roll);
                break;
            case 4:
                printf("Enter roll number to delete: ");
                scanf("%d", &roll);
                deleteRoll(roll);
                break;
            case 5:
                display();
                break;
            case 6:
                printf("Exiting program.\n");
                exit(0);
            default:
                printf("Invalid choice! Please choose between 1 and 6.\n");
        }
    }
    return 0;
}
```

OUTPUT:

```
PS E:\cpp programs> cd "e:\cpp programs\" ; if ($?) { gcc test.c -o test } ; if ($?) { .\test }

--- Student Roll Number Management Menu ---
1. Insert at Beginning
2. Insert at End
3. Search Roll Number
4. Delete Roll Number
5. Display List
6. Exit
Enter your choice:

                  2
Enter roll number to insert at end: 101
Inserted 101 at the end.
Current Roll Numbers: 101 -> NULL

--- Student Roll Number Management Menu ---
1. Insert at Beginning
2. Insert at End
3. Search Roll Number
4. Delete Roll Number
5. Display List
6. Exit
Enter your choice: 2
Enter roll number to insert at end: 103
Inserted 103 at the end.
Current Roll Numbers: 101 -> 103 -> NULL

--- Student Roll Number Management Menu ---
1. Insert at Beginning
2. Insert at End
3. Search Roll Number
4. Delete Roll Number
5. Display List
```

```
--- Student Roll Number Management Menu ---
1. Insert at Beginning
2. Insert at End
3. Search Roll Number
4. Delete Roll Number
5. Display List
6. Exit
Enter your choice: 1
Enter roll number to insert at beginning: 100
Inserted 100 at the beginning.
Current Roll Numbers: 100 -> 101 -> 103 -> NULL

--- Student Roll Number Management Menu ---
1. Insert at Beginning
2. Insert at End
3. Search Roll Number
4. Delete Roll Number
5. Display List
6. Exit
Enter your choice: 3
Enter roll number to search: 101
Roll number 101 found at position 2 in the list.

--- Student Roll Number Management Menu ---
1. Insert at Beginning
2. Insert at End
3. Search Roll Number
4. Delete Roll Number
5. Display List
6. Exit
Enter your choice: 3
Enter roll number to search: 999
Roll number 999 is not available in the list.

--- Student Roll Number Management Menu ---
1. Insert at Beginning
2. Insert at End
3. Search Roll Number
```

```
--- Student Roll Number Management Menu ---
1. Insert at Beginning
2. Insert at End
3. Search Roll Number
4. Delete Roll Number
5. Display List
6. Exit
Enter your choice: 4
Enter roll number to delete: 101
Deleted roll number 101 from the list.
Current Roll Numbers: 100 -> 103 -> NULL

--- Student Roll Number Management Menu ---
1. Insert at Beginning
2. Insert at End
3. Search Roll Number
4. Delete Roll Number
5. Display List
6. Exit
Enter your choice: 6
Exiting program.
```

Q2) Develop a C program for a Doubly Linked List representing a sequence of web pages visited by a user. The program should insert a new page, move forward and backward, delete a specified page, and display the pages from first-to-last and last-to-first while handling beginning and end conditions correctly.

SOL)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
   char url[100];
   struct Node* prev;
   struct Node* next;
};

struct Node* head = NULL;
struct Node* current = NULL;

void displayForward() {
   struct Node* temp = head;
   if (temp == NULL) {
      printf("History is empty.\n");
      return;
   }
   printf("History (First to Last): ");
   while (temp != NULL) {
      if (temp == current) {
         printf("[Current: %s] <-> ", temp->url);
      } else {
         printf("%s <-> ", temp->url);
      }
      temp = temp->next;
   }
   printf("NULL\n");
}
```

```
void displayBackward() {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("History is empty.\n");
        return;
    }
    while (temp->next != NULL) {
        temp = temp->next;
    }
    printf("History (Last to First): ");
    while (temp != NULL) {
        if (temp == current) {
            printf("[Current: %s] <-> ", temp->url);
        } else {
            printf("%s <-> ", temp->url);
        }
        temp = temp->prev;
    }
    printf("NULL\n");
}

void visitPage(char* url) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    strcpy(newNode->url, url);
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
        current = newNode;
    } else {
        struct Node* temp = current->next;
        while (temp != NULL) {
            struct Node* nextNode = temp->next;
            free(temp);
            temp = nextNode;
        }
        current->next = newNode;
        newNode->prev = current;
        current = newNode;
    }
    printf("Visited: %s\n", url);
    displayForward();
}

void moveBack() {
    if (current != NULL && current->prev != NULL) {
        current = current->prev;
        printf("Moved backward to: %s\n", current->url);
    } else {
        printf("You are already at the first page. Cannot go back further.\n");
    }
    displayForward();
}

void moveForward() {
    if (current != NULL && current->next != NULL) {
        current = current->next;
        printf("Moved forward to: %s\n", current->url);
    } else {
```

```
        printf("You are already at the last page. Cannot go forward further.\n");
    }
    displayForward();
}

void deletePage(char* url) {
    struct Node* temp = head;
    while (temp != NULL && strcmp(temp->url, url) != 0) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Page '%s' not found in history.\n", url);
        return;
    }

    if (temp == head) {
        head = temp->next;
        if (head != NULL) {
            head->prev = NULL;
        }
    } else {
        temp->prev->next = temp->next;
        if (temp->next != NULL) {
            temp->next->prev = temp->prev;
        }
    }

    if (current == temp) {
        if (temp->next != NULL) {
            current = temp->next;
        } else if (temp->prev != NULL) {
            current = temp->prev;
        } else {
            current = NULL;
        }
    }

    free(temp);
    printf("Deleted page '%s' from history.\n", url);
    displayForward();
}

int main() {
    int choice;
    char url[100];

    while (1) {
        printf("\n--- Browser History Management Menu ---\n");
        printf("1. Visit New Page\n");
        printf("2. Move Backward\n");
        printf("3. Move Forward\n");
        printf("4. Delete Specific Page\n");
        printf("5. Display Forward (First to Last)\n");
        printf("6. Display Backward (Last to First)\n");
        printf("7. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Exiting.\n");
            break;
```

```
        }

        switch (choice) {
            case 1:
                printf("Enter URL to visit: ");
                scanf("%s", url);
                visitPage(url);
                break;
            case 2:
                moveBack();
                break;
            case 3:
                moveForward();
                break;
            case 4:
                printf("Enter URL to delete: ");
                scanf("%s", url);
                deletePage(url);
                break;
            case 5:
                displayForward();
                break;
            case 6:
                displayBackward();
                break;
            case 7:
                printf("Exiting program.\n");
                exit(0);
            default:
                printf("Invalid choice! Please choose between 1 and 7.\n");
        }
    }
    return 0;
}
```

OUTPUT:

```
PS E:\cpp programs> cd "e:\cpp programs\" ; if ($?) { gcc test.c -o test } ; if ($?) { .\test }

--- Browser History Management Menu ---
1. Visit New Page
2. Move Backward
3. Move Forward
4. Delete Specific Page
5. Display Forward (First to Last)
6. Display Backward (Last to First)
7. Exit
Enter your choice: 1
Enter URL to visit: GOOGLE.COM
Visited: GOOGLE.COM
History (First to Last): [Current: GOOGLE.COM] <-> NULL

--- Browser History Management Menu ---
1. Visit New Page
2. Move Backward
3. Move Forward
4. Delete Specific Page
5. Display Forward (First to Last)
6. Display Backward (Last to First)
7. Exit
Enter your choice: 1
Enter URL to visit: GITHUB.COM
Visited: GITHUB.COM
History (First to Last): GOOGLE.COM <-> [Current: GITHUB.COM] <-> NULL

--- Browser History Management Menu ---
1. Visit New Page
2. Move Backward
3. Move Forward
4. Delete Specific Page
5. Display Forward (First to Last)
6. Display Backward (Last to First)
7. Exit
Enter your choice: 2
Moved backward to: GOOGLE.COM
History (First to Last): [Current: GOOGLE.COM] <-> GITHUB.COM <-> NULL
```

```
--- Browser History Management Menu ---
1. Visit New Page
2. Move Backward
3. Move Forward
4. Delete Specific Page
5. Display Forward (First to Last)
6. Display Backward (Last to First)
7. Exit
Enter your choice: 6
History (Last to First): GITHUB.COM <-> [Current: GOOGLE.COM] <-> NULL

--- Browser History Management Menu ---
1. Visit New Page
2. Move Backward
3. Move Forward
4. Delete Specific Page
5. Display Forward (First to Last)
6. Display Backward (Last to First)
7. Exit
Enter your choice: 7
Exiting program.
```

UNIT-4

Q1) A system stores unique integer identification numbers using a Binary Search Tree. Write a C program to insert n values, display inorder, preorder and postorder traversals, search for a specified value, and report whether it exists. Use the output to explain why inorder traversal produces sorted values.

SOL)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left;
   struct Node *right;
};

struct Node* createNode(int value) {
   struct Node *newNode;
   newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}

struct Node* insert(struct Node *root, int value) {
   if (root == NULL)
      return createNode(value);
   if (value < root->data)
      root->left = insert(root->left, value);
   else if (value > root->data)
      root->right = insert(root->right, value);
   return root;
}

void inorder(struct Node *root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

void preorder(struct Node *root) {
   if (root != NULL) {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

void postorder(struct Node *root) {
   if (root != NULL) {
      postorder(root->left);
      postorder(root->right);
      printf("%d ", root->data);
   }
}

int search(struct Node *root, int key) {
   if (root == NULL)
```

```
        return 0;
    if (root->data == key)
        return 1;
    if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}

int main() {
    struct Node *root = NULL;
    int n, value, key, i;

    printf("Enter number of values: ");
    scanf("%d", &n);
    printf("Enter %d unique values:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("\nInorder Traversal: ");
    inorder(root);
    printf("\nPreorder Traversal: ");
    preorder(root);
    printf("\nPostorder Traversal: ");
    postorder(root);

    printf("\n\nEnter value to search: ");
    scanf("%d", &key);
    if (search(root, key))
        printf("%d exists in the BST.\n", key);
    else
        printf("%d does not exist in the BST.\n", key);

    return 0;
}
```

OUTPUT:

```
PS E:\cpp programs> cd "e:\cpp programs\" ; if ($?) { gcc test.c -o test } ; if ($?) { .\test }
Enter number of values: 7
Enter 7 unique values:
50 30 70 20 40 60 80

Inorder Traversal: 20 30 40 50 60 70 80
Preorder Traversal: 50 30 20 40 70 60 80
Postorder Traversal: 20 40 30 60 80 70 50

Enter value to search: 60
60 exists in the BST.
```

Q2) Extend a Binary Search Tree program to support deletion. The program should create a BST, delete a user-specified node, correctly handle nodes with zero, one and two children, and display inorder traversal before and after deletion. Test the program separately for all three deletion cases.

SOL)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
```

```
};

struct Node* createNode(int value) {
   struct Node *newNode;
   newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}

struct Node* insert(struct Node *root, int value) {
   if (root == NULL)
      return createNode(value);
   if (value < root->data)
      root->left = insert(root->left, value);
   else if (value > root->data)
      root->right = insert(root->right, value);
   return root;
}

void inorder(struct Node *root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

struct Node* findMin(struct Node *root) {
   while (root->left != NULL)
      root = root->left;
   return root;
}

struct Node* deleteNode(struct Node *root, int value) {
   struct Node *temp;
   if (root == NULL)
      return root;

   if (value < root->data)
      root->left = deleteNode(root->left, value);
   else if (value > root->data)
      root->right = deleteNode(root->right, value);
   else {
      if (root->left == NULL && root->right == NULL) {
         free(root);
         return NULL;
      } else if (root->left == NULL) {
         temp = root->right;
         free(root);
         return temp;
      } else if (root->right == NULL) {
         temp = root->left;
         free(root);
         return temp;
      } else {
         temp = findMin(root->right);
         root->data = temp->data;
         root->right = deleteNode(root->right, temp->data);
```

```
        }
    }
    return root;
}

int main() {
    struct Node *root = NULL;
    int n, value, key, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter values:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("\nInorder before deletion: ");
    inorder(root);

    printf("\nEnter node to delete: ");
    scanf("%d", &key);
    root = deleteNode(root, key);

    printf("\nInorder after deletion: ");
    inorder(root);

    return 0;
}
```

OUTPUT:

```
PS E:\cpp programs> cd "e:\cpp programs\" ; if ($?) { gcc test.c -o test } ; if ($?) { .\test }
Enter number of nodes: 7
Enter values:
50 30 70 20 40 60 80

Inorder before deletion: 20 30 40 50 60 70 80
Enter node to delete: 20

Inorder after deletion: 30 40 50 60 70 80
```